

Python Programming

Unit 05 – Lecture 02 Notes

Members, Class Attributes, GC, Abstract Classes

Tofik Ali

February 17, 2026

Contents

1	Lecture Overview	1
2	Core Concepts	1
2.1	Public and Private Members (Python Style)	1
2.2	Built-in Class Attributes	2
2.3	Garbage Collection (High-Level)	2
2.4	Abstract Classes (ABC)	2
3	Demo Walkthrough	2
4	Interactive Checkpoints (with Solutions)	3
5	Practice Exercises (with Solutions)	3
6	Exit Question (with Solution)	3

1 Lecture Overview

This lecture covers three practical topics used in real code:

- how Python represents “public/private” members (mostly conventions),
- how to inspect classes using built-in attributes,
- and how to design abstract classes using `abc`.

2 Core Concepts

2.1 Public and Private Members (Python Style)

Python does not enforce strict access modifiers like some languages. Instead it uses naming conventions:

- `value`: public

- `_value`: internal use (“protected” by convention)
- `__value`: name mangling (reduces accidental access)

Name Mangling

If a class defines `__balance`, Python internally renames it to include the class name. This helps prevent accidental access from subclasses.

```
class Account:
    def __init__(self):
        self.__balance = 0
```

Access is still possible (not truly private), but it is discouraged:

```
a = Account()
print(a._Account__balance)
```

2.2 Built-in Class Attributes

Some useful built-in attributes:

- `__dict__`: attribute dictionary (what the object/class contains)
- `__doc__`: documentation string (docstring)
- `__class__`: class of an object
- `__module__`: module name

```
class A:
    """Example class"""
    x = 10

print(A.__doc__)
print(A.__dict__)
```

2.3 Garbage Collection (High-Level)

Python manages memory automatically:

- reference counting frees objects when reference count becomes 0,
- cycle detector handles circular references.

In most cases, you do not need to manually free memory. Avoid relying on `__del__` for critical logic, because its timing is not guaranteed.

2.4 Abstract Classes (ABC)

An abstract class defines an interface. Subclasses must implement abstract methods.

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        pass
```

3 Demo Walkthrough

File: demo/abstract_shape_demo.py

The demo shows:

- Shape cannot be instantiated (because area is abstract),
- Circle and Rectangle implement area().

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: what does name mangling do?

Answer: It renames `__attr` internally to include the class name, reducing accidental access and collisions in subclasses.

Checkpoint 2 Solution

Question: why use abstract classes?

Answer: To enforce that subclasses implement required methods and to define a clean interface.

5 Practice Exercises (with Solutions)

Exercise 1: Private Balance

Task: Create an Account class with `__balance` and methods `deposit` and `get_balance`.

Solution:

```
class Account:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Exercise 2: Abstract Class

Task: Create abstract class `Vehicle` with abstract method `start()`.

Solution:

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
```

6 Exit Question (with Solution)

Question: which module is used for abstract base classes?

Answer: abc

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Use public/private naming conventions in Python classes
- Inspect built-in class attributes like `__dict__` and `__doc__`
- Explain garbage collection at a high level
- Create abstract classes using the `abc` module

Public vs Private in Python

- Python uses conventions (not strict access control)
- `name`: public
- `_name`: internal (“protected” by convention)
- `__name`: name-mangled (harder to access accidentally)

Built-in Class Attributes

- `__dict__`: attributes dictionary
- `__doc__`: documentation string
- `__class__`: object’s class
- `__module__`: module name where class is defined

Garbage Collection (GC)

- Automatically frees memory of unreachable objects
- Python uses reference counting + cycle detection
- Usually you do not manually free memory

Abstract Classes

- Abstract class defines an interface (methods to implement)
- Prevents creating incomplete objects

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Demo: Abstract Shape

- File: demo/abstract_shape_demo.py
- Implements:
 - abstract base class Shape
 - derived Circle and Rectangle

Checkpoint 1

Question: What does name mangling mean for an attribute like `__balance`?

Checkpoint 2

Question: Why do we use abstract classes?

Think-Pair-Share

Discuss:

- Create an abstract class `Vehicle` with method `start()`.
- What subclasses can implement it?

Key Takeaways

- “Private” in Python is mostly convention (`_` and `__`)
- Built-in attributes help introspection (`__dict__`, `__doc__`)
- GC manages memory automatically
- Abstract classes define required methods for subclasses

Exit Question

Which module is used for abstract base classes in Python?